

Personalized Networking Assistant

Project Description

The Personalized Networking Assistant is an AI-powered web application that leverages Generative Artificial Intelligence to help professionals, students, entrepreneurs, and job seekers build meaningful connections during networking events. The platform analyzes event descriptions and user profiles to generate personalized conversation starters, suggest relevant discussion topics, verify facts through Wikipedia, and maintain conversation history for future reference.

The application consists of several intelligent modules, including the Event Analyzer, which analyzes user biographies and event descriptions to generate personalized networking suggestions.

The Topic Generator, which produces relevant networking topics; the Conversation Starter Generator, which creates personalized opening statements; the Wikipedia Fact Checker, which validates generated information for accuracy; the Conversation History Logger, which stores previous networking sessions; and the Feedback Logger, which records user feedback to improve future interactions.

Utilizing the Google Gemini API, the Personalized Networking Assistant delivers context-aware, natural, and engaging networking suggestions in real time. The application is developed using Python, FastAPI, and Streamlit, while SQLite is used for storing user profiles, networking sessions, conversation history, and feedback. The integration of the Wikipedia API ensures factual accuracy by verifying AI-generated content before presenting it to users.

The application follows secure API key management practices through environment variables and provides a simple, responsive, and user-friendly interface that helps users confidently initiate conversations in professional environments.

Scenarios

Scenario 1

A university student attends a technology conference and enters the event description along with a short personal profile. The Personalized Networking Assistant analyzes the event, identifies key discussion themes such as Artificial Intelligence and Cloud Computing, generates personalized conversation starters, and suggests networking topics relevant to the student's interests. Before displaying the results, the system verifies technical facts using Wikipedia to improve reliability.

Scenario 2

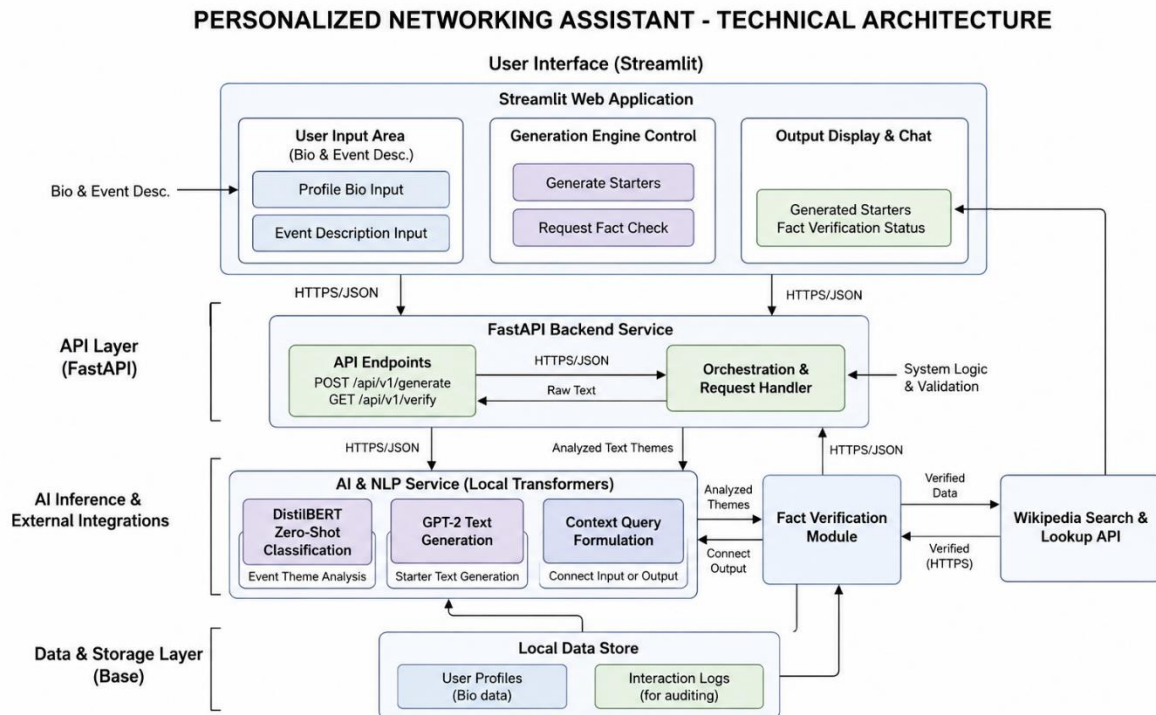
A software engineer participates in a startup networking event. The application analyzes the event description and generates conversation starters focused on entrepreneurship, software development, and innovation. The user receives relevant networking topics and verified information that helps initiate professional discussions with startup founders and investors.

Scenario 3

A marketing professional attends a digital marketing summit. The Personalized Networking Assistant analyzes the event agenda, generates personalized opening conversations,

recommends trending marketing topics, verifies industry-related facts using Wikipedia, and stores the interaction history for future reference. After the event, the user submits feedback, which is securely stored for future improvements.

Technical Architecture:



Description:

The Personalized Networking Assistant utilizes a modular three-tier architecture to provide intelligent, AI-powered networking assistance. The Presentation Layer consists of a responsive web interface developed using Streamlit, allowing users to enter event details, view AI-generated conversation starters, explore suggested networking topics, verify facts, and access conversation history through an intuitive interface.

The Application Layer is implemented using FastAPI, which manages API routing, user request processing, event analysis, topic generation, conversation starter generation, Wikipedia fact verification, conversation history management, and feedback logging. The backend securely communicates with the Google Gemini API to generate personalized networking suggestions and with the Wikipedia API to verify factual information.

The Data Layer uses SQLite to store user profiles, event information, networking sessions, generated conversation starters, fact-check results, conversation history, and user feedback. Secure API key management is handled using environment variables (.env), ensuring safe access to external AI services while maintaining application security and scalability.

Pre-requisites

Before getting started, ensure you have a basic familiarity with cloud computing and version control (Git) will be helpful. Also, install Visual Studio Code for writing and managing code efficiently.

Python 3.11+: Core programming language used to build the application backend, manage logic, handle APIs, and run AI/NLP processing modules.

<https://www.python.org/downloads/>

FastAPI Documentation: High-performance Python web framework used to create REST APIs for communication between the frontend, AI services, and verification modules.

<https://fastapi.tiangolo.com/>

Streamlit Documentation: Interactive frontend framework used to build the web-based user interface for profile input, event descriptions, and displaying generated networking starters.

<https://docs.streamlit.io/>

Hugging Face Transformers: AI/ML library providing pre-trained NLP models such as DistilBERT and GPT-2 for text classification, theme analysis, and starter generation.

<https://huggingface.co/docs/transformers/>

Wikipedia API (Python): External verification service used to fetch and validate factual information from Wikipedia sources in real time.

<https://pypi.org/project/wikipedia/>

Git Version Control: Source code management system used for tracking project changes, collaboration, version history, and deployment workflows.

<https://git-scm.com/doc>

Virtual Environment (venv): Python environment isolation tool used to manage project dependencies separately and maintain consistent package versions across development setups.

<https://docs.python.org/3/library/venv.html>

Project Workflow

Epic 1: Model Selection and Architecture

- **Story 1:** Model Research and Selection
- **Story 2:** Defining the Application Architecture
- **Story 3:** Environment Setup

Epic 2: Core Functionalities Development

- **Story 1:** Data Schema Definition
- **Story 2:** Event Analyzer Service Development
- **Story 3:** Topic Generator Service Development

- **Story 4:** Fact Checker Service Development
- **Story 5:** History Logger Service Development
- **Story 6:** Feedback Logger Service Development
- **Story 7:** API Routes Implementation
- **Story 8:** Application Entry Point Configuration

Epic 3: Main Application Logic

- **Story 1:** Centralized Routing Architecture
- **Story 2:** Service Layer Design Principles
- **Story 3:** FastAPI-Specific Features Integration

Epic 4: Frontend UI Using Streamlit

- **Story 1:** Application Setup and Configuration
- **Story 2:** Input Section and Main Generation Flow
- **Story 3:** Results Display and Feedback System
- **Story 4:** Fact-Checking Section
- **Story 5:** Conversation History View
- **Story 6:** Feedback History View
- **Story 7:** Streamlit State Management Implementation

Epic 5: Testing and Local Deployment

- **Story 1:** Testing Philosophy and Framework Selection
- **Story 2:** Event Analyzer Service Testing
- **Story 3:** Topic Generator Service Testing
- **Story 4:** Fact Checker Service Testing
- **Story 5:** API Routes Testing with httpx TestClient
- **Story 6:** Running Tests and Interpreting Results
- **Story 7:** Local Deployment
- **Story 8:** Manual Testing and Verification

Epic1: Model Selection and Architecture

This milestone focuses on selecting the most appropriate Artificial Intelligence model and designing a scalable architecture for the Personalized Networking Assistant. The selected model should analyze event descriptions, identify relevant networking domains, and support intelligent networking suggestions. The architecture clearly defines the interaction between the

Streamlit frontend, FastAPI backend, Google Gemini API model, Wikipedia API, and SQLite database to ensure efficient, reliable, and maintainable application development.

Set Up the AI Model Environment

Before the Personalized Networking Assistant can analyze event descriptions, the required Python libraries and AI model must be installed and configured. The application uses the Google Gemini API to analyze event descriptions, understand user interests, generate personalized networking suggestions, and produce intelligent conversation starters.

The following setup steps were completed:

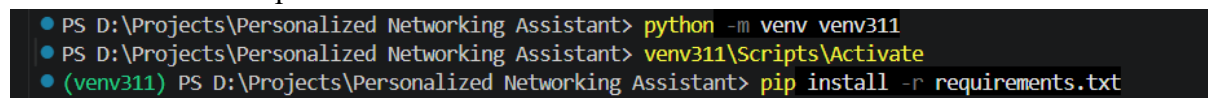
- Install Python 3.10 or above.
- Create a Python virtual environment.
- Install all required dependencies using the requirements.txt file.
- Configure the project directory structure.
- Verify successful installation of all required libraries.

Create a virtual environment:

- `python -m venv venv311`

Activate the virtual environment:

- `venv311\Scripts\activate`

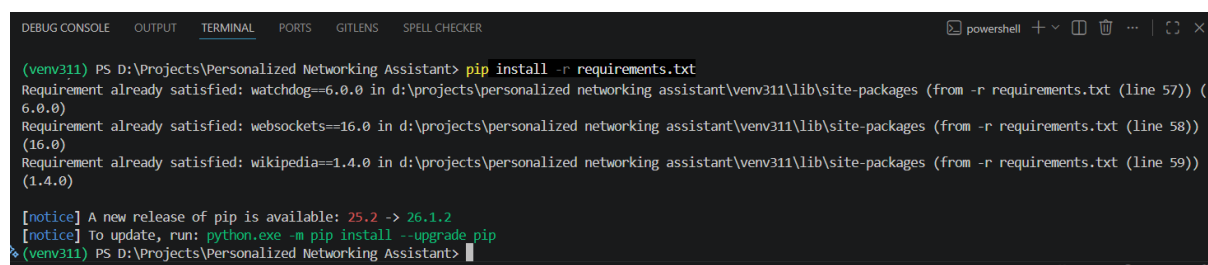


```

PS D:\Projects\Personalized Networking Assistant> python -m venv venv311
PS D:\Projects\Personalized Networking Assistant> venv311\Scripts\Activate
(venv311) PS D:\Projects\Personalized Networking Assistant> pip install -r requirements.txt
  
```

Install project dependencies:

- `pip install -r requirements.txt`



```

(venv311) PS D:\Projects\Personalized Networking Assistant> pip install -r requirements.txt
Requirement already satisfied: watchdog==6.0.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 57)) (6.0.0)
Requirement already satisfied: websockets==16.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 58)) (16.0)
Requirement already satisfied: wikipedia==1.4.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 59)) (1.4.0)
[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv311) PS D:\Projects\Personalized Networking Assistant>
  
```

Figure 1.1: Project Environment Setup

• Story 1: Model Research and Selection

Selecting the appropriate Artificial Intelligence model is an important step in developing the Personalized Networking Assistant. The model should understand event descriptions, generate personalized conversation starters, recommend networking topics, and produce natural language responses.

The following activities were performed:

- Analyze the project requirements.
- Explore Google Gemini language models.
- Compare available Gemini models based on response quality and speed.

- Evaluate model performance for conversational AI tasks.
- Select the most suitable model.

After evaluation, the Google Gemini 1.5 Flash model was selected because it provides:

- Fast response generation
- High-quality natural language generation
- Strong contextual understanding
- Low latency for real-time applications
- Easy integration using the Google Generative AI Python SDK
- Suitable for conversational AI and networking assistance

The selected model generates personalized networking suggestions, conversation starters, and discussion topics based on the user's profile and event description.

Story 2: Defining the Application Architecture

The Personalized Networking Assistant follows a modular three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

The architecture includes the following components:

- **Presentation Layer:** Streamlit user interface.
- **Application Layer:** FastAPI backend, Google Gemini API model, and Wikipedia API.
- **Data Layer:** SQLite database.

The Streamlit frontend collects the user biography and event description. These requests are sent to the FastAPI backend, which validates the input, sends it to the Google Gemini API for personalized conversation generation, verifies important facts using the Wikipedia API, stores conversation history and feedback in the SQLite database, and returns the generated results to the user interface.

The architecture is designed using a modular service-based approach. The FastAPI backend separates different functionalities into individual services such as the Event Analyzer, Topic Generator, Conversation Starter Generator, Wikipedia Fact Checker, Conversation History Logger, and Feedback Logger. This modular design improves maintainability, scalability, and future enhancement of the application.

Architecture Workflow

- User enters biography and event description through Streamlit.
- FastAPI validates the user input.
- Google Gemini API generates personalized conversation starters and networking suggestions.
- Wikipedia API verifies important facts.

- SQLite stores conversation history and feedback.
- Results are returned and displayed in the Streamlit interface.

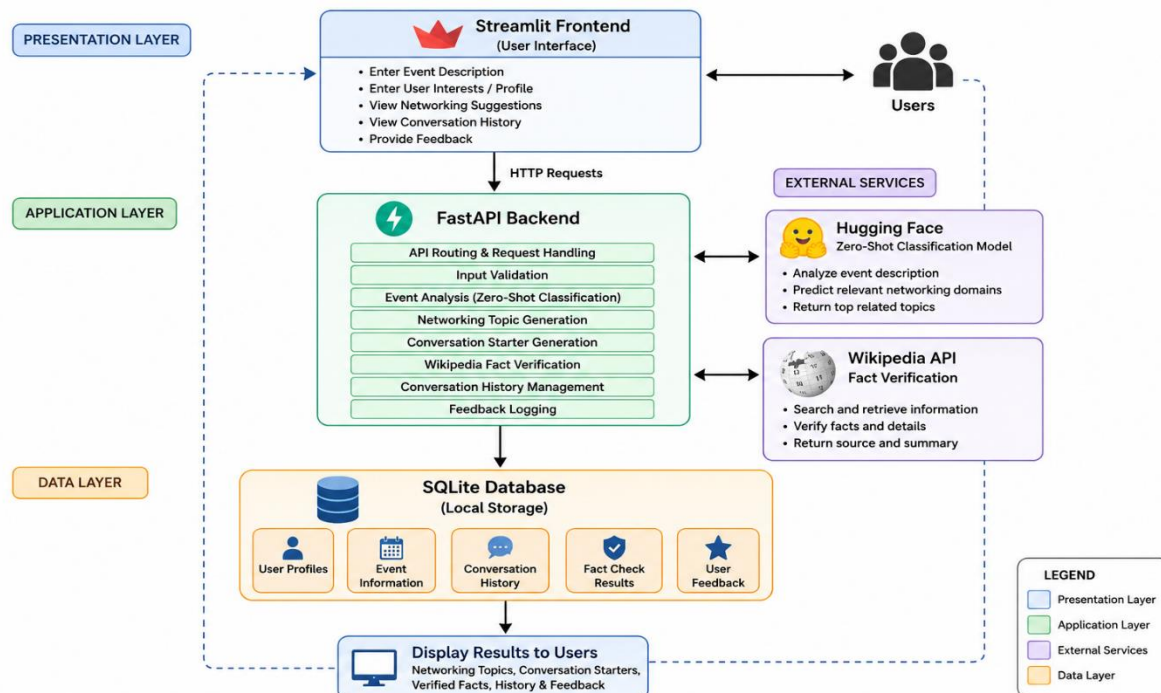


Figure 1.3: Solution Architecture of the Personalized Networking Assistant

Story 3: Environment Setup

The development environment was configured to support the Personalized Networking Assistant using Python, FastAPI, Streamlit, SQLite, and Google Gemini.

The following setup steps were completed:

- Install Python 3.10 or above.
- Create a virtual environment.
- Install the Google Generative AI Python SDK.
- Configure the Google Gemini API key securely using the `.env` environment file.
- Create the project directory structure.
- Verify successful application execution.

Create a virtual environment:

- `python -m venv venv311`

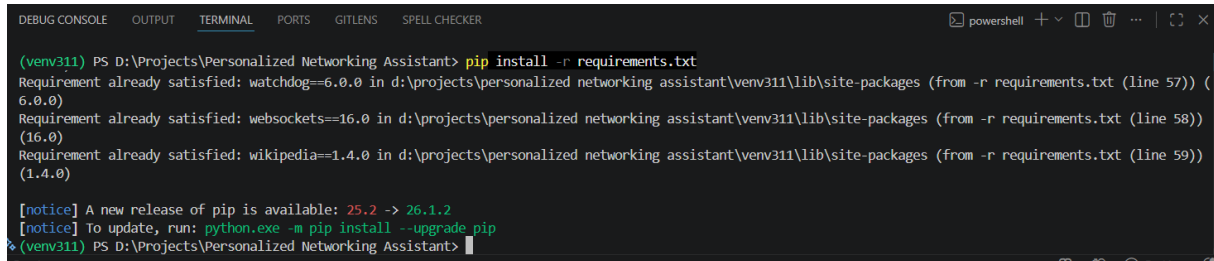
Activate the virtual environment:

- `venv311\Scripts\activate`

```
PS D:\Projects\Personalized Networking Assistant> python -m venv venv311
PS D:\Projects\Personalized Networking Assistant> venv311\Scripts\Activate
(venv311) PS D:\Projects\Personalized Networking Assistant> pip install -r requirements.txt
```

Install project dependencies:

- `pip install -r requirements.txt`
- `pip install google-generativeai`



```

DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS  GITLENS  SPELL CHECKER
(venv311) PS D:\Projects\Personalized Networking Assistant> pip install -r requirements.txt
Requirement already satisfied: watchdog==6.0.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 57)) (6.0.0)
Requirement already satisfied: websockets==16.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 58)) (16.0)
Requirement already satisfied: wikipedia==1.4.0 in d:\projects\personalized networking assistant\venv311\lib\site-packages (from -r requirements.txt (line 59)) (1.4.0)

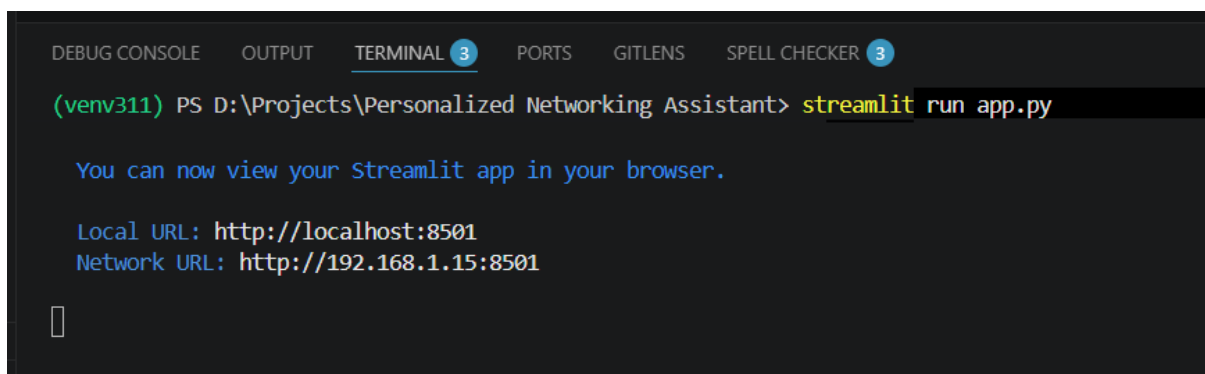
[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv311) PS D:\Projects\Personalized Networking Assistant>
  
```

Run the FastAPI backend:

- uvicorn main:app --reload

Run the Streamlit frontend:

- streamlit run streamlit_app.py



```

DEBUG CONSOLE  OUTPUT  TERMINAL 3  PORTS  GITLENS  SPELL CHECKER 3
(venv311) PS D:\Projects\Personalized Networking Assistant> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.15:8501

[]
  
```

Figure 1.4: Development Environment Setup

Epic2: Core Functionalities Development

Story 1: Data Schema Definition

The SQLite database schema was designed to efficiently store user information, event descriptions, networking topics, conversation history, verified facts, and user feedback. Separate tables were created to maintain data consistency and simplify database operations. Primary keys uniquely identify each record, while timestamps are stored to maintain historical information. This database structure supports efficient retrieval of previous conversations and improves application scalability.

Database Tables

The SQLite database contains the following tables:

- **users** – Stores user biography and profile details.
- **events** – Stores event descriptions submitted by users.
- **conversation_history** – Stores generated conversation starters and networking topics.
- **feedback** – Stores user feedback and ratings.
- **fact_checks** – Stores Wikipedia verification results.

Story 2: Event Analyzer Service Development

The Personalized Networking Assistant consists of several AI-powered modules that work together to provide users with meaningful networking support during professional events.

- **Event Analyzer: Analyzes the event description using the Google Gemini API.**
- The service utilizes the DistilBERT model to perform theme identification and categorization. Extracted themes help the system understand the event's focus areas and serve as input for conversation generation.
- The modular design of the Event Analyzer allows future enhancements such as sentiment analysis, keyword extraction, and advanced topic classification.

Story 3: Topic Generator Service Development

- **Topic Generator:** Suggests relevant networking topics that users can discuss during professional interactions.
- The service processes event information and user preferences to generate context-aware conversation prompts. Generated suggestions encourage professional engagement and help users build connections more confidently.
- The service is designed to be extensible, allowing integration of advanced language models for improved conversational quality in future versions.
- **Conversation Starter Generator:** Generates personalized networking suggestions using Google Gemini.

Story 4: Fact Checker Service Development

- **Wikipedia Fact Checker:** Verifies important facts and information using the Wikipedia API to improve the reliability of AI-generated responses.
- The Fact Checker Service provides quick verification of topics, technologies, organizations, and concepts related to networking events. This feature helps users gain confidence before participating in discussions.
- This functionality supports informed networking by giving users access to trusted background information in real time.

Story 5: History Logger Service Development

- **Conversation History Logger:** Stores previously generated conversation starters and networking suggestions in the SQLite database for future reference.
- Users can review past networking suggestions through the application's history section. This feature helps users revisit useful conversation ideas and track their networking preparation over time.
- The service uses JSON-based storage for simplicity and efficient retrieval.

Story 6: Feedback Logger Service Development

- **Feedback Logger:** Records user feedback regarding the quality and usefulness of AI-generated responses to support future improvements.
- Collected feedback is stored in a dedicated JSON file and can be analyzed to improve future recommendation quality. The feedback mechanism enables continuous enhancement of user experience and personalization capabilities.
- This service plays an important role in creating a user-centric networking assistant.


Personalized Networking Assistant

Generate smart conversation starters for networking events.

Event Description

Your Interests

Generate


Conversation History

Event: AI Conference

Interests: Machine Learning

Event: Google Cloud Summit

Interests: Artificial Intelligence

Event: AI Conference

Interests: Cloud Computing

Figure 2.1: Personalized Networking Assistant User Interface

Story 7: API Routes Implementation

Define API Routes

- Create FastAPI routes for handling event analysis, conversation starter generation, topic generation, fact checking, conversation history, and feedback management.
- Define separate endpoints for each service to maintain a modular and scalable backend architecture.

Implemented API Endpoints

The FastAPI backend exposes the following REST API endpoints:

Endpoint	Method	Purpose
/generate	POST	Generate networking suggestions
/history	GET	Retrieve conversation history
/feedback	POST	Save user feedback
/fact-check	POST	Verify information using Wikipedia

Process User Input

- Design the Streamlit interface to collect user profile information and event descriptions.
- Validate all user inputs before processing.

- Ensure that required fields are not empty and that input lengths remain within acceptable limits.
- Convert validated user input into structured requests before sending them to the backend services.

Integrate AI Models

- Analyze the user biography and event description using the Google Gemini API.
- Generate personalized networking topics based on the user's interests and event description.
- Generate personalized conversation starters, networking topics, and discussion ideas using Google Gemini.
- Return AI-generated results to the FastAPI backend.

Integrate Wikipedia API

- Send generated topics and factual statements to the Wikipedia API.
- Verify important information before displaying it to the user.
- Include verified references whenever applicable.

Database Integration

- Store user profiles, event details, conversation history, generated networking suggestions, fact-checking results, and user feedback in the SQLite database.
- Retrieve historical conversations and feedback whenever requested by the user.
- The SQLite database automatically stores every generated networking session along with timestamps to support future retrieval.

Return Results

- Display AI-generated conversation starters and networking topics through the Streamlit interface.
- Present fact-checking results and conversation history in a clear and user-friendly format.
- Allow users to submit feedback for continuous improvement of the application.
- Display appropriate success and error messages based on API responses.

Event: Google Cloud Generative AI Workshop

Interests: Artificial Intelligence and Cloud Computing

Wikipedia Fact Checker

Enter a topic

Artificial Intelligence

Fact Check

Feedback

 Helpful

 Not Helpful

Thanks for your feedback!

Figure 2.2: Wikipedia Fact Checker and User Feedback Interface

Application Entry Point

The FastAPI application entry point initializes all API routes, loads the AI model, establishes the SQLite database connection, and configures middleware required for request processing. The main application file acts as the central interface through which all frontend requests are processed.

Story 8: Application Entry Point Configuration

The FastAPI application entry point (main.py) initializes the backend server, loads environment variables, configures middleware, registers API routes, establishes the SQLite database connection, and starts all backend services required by the Personalized Networking Assistant. This component ensures that all backend services are properly connected and available to frontend requests. Configuration settings, model loading procedures, and route registration are handled during application startup.

A centralized entry point improves project organization and simplifies deployment and maintenance activities.

Epic 3: Main Application Logic

Epic 3 focuses on implementing the core application logic of the Personalized Networking Assistant. The main application integrates the Streamlit frontend with the FastAPI backend, Google Gemini API, Wikipedia API, and SQLite database. It processes user requests, generates AI-powered networking suggestions, verifies information, stores conversation history, and displays the results to users.

Story 1: Centralized Routing Architecture

The application follows a centralized routing architecture where all incoming HTTP requests are processed through the FastAPI backend. Each API endpoint is responsible for a specific functionality, ensuring modularity and maintainability.

Story 2: Service Layer Design Principles

The service layer separates business logic from API routing. Individual services handle event analysis, networking topic generation, Wikipedia fact verification, conversation history management, and feedback storage. This modular design simplifies maintenance and future enhancements.

Story 3: FastAPI-Specific Features Integration

FastAPI features such as automatic request validation, asynchronous request handling, dependency injection, and interactive API documentation are integrated to improve application performance and developer productivity.

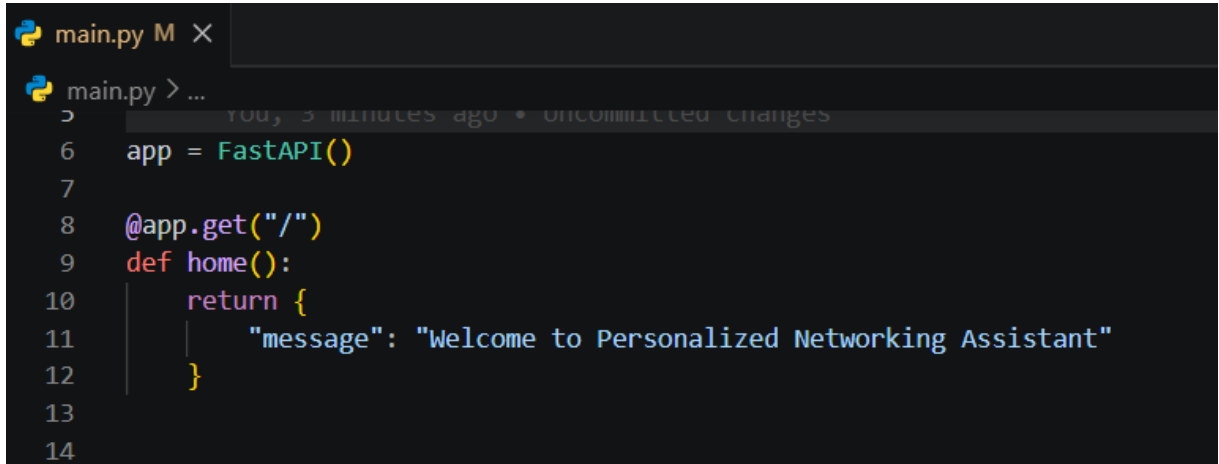
Develop the Main Application Logic

Define the Main Application Workflow

The main application performs the following operations:

- Accept user profile information and event description through the Streamlit interface.
- Validate user inputs before processing.
- Send the validated request to the FastAPI backend.
- Generate personalized conversation starters and networking topics using Google Gemini.

- Verify factual information using the Wikipedia API.
- Store user interactions and conversation history in the SQLite database.
- Display the generated results through the Streamlit interface.



```
main.py M X
main.py > ...
5
6 app = FastAPI()
7
8 @app.get("/")
9 def home():
10     return {
11         "message": "Welcome to Personalized Networking Assistant"
12     }
13
14
```

Figure 3.1: FastAPI Main Application Logic

Handle User Input

- Receive the user's biography and event description.
- Validate that both input fields are provided before processing.
- Forward the validated input to the AI analysis module.

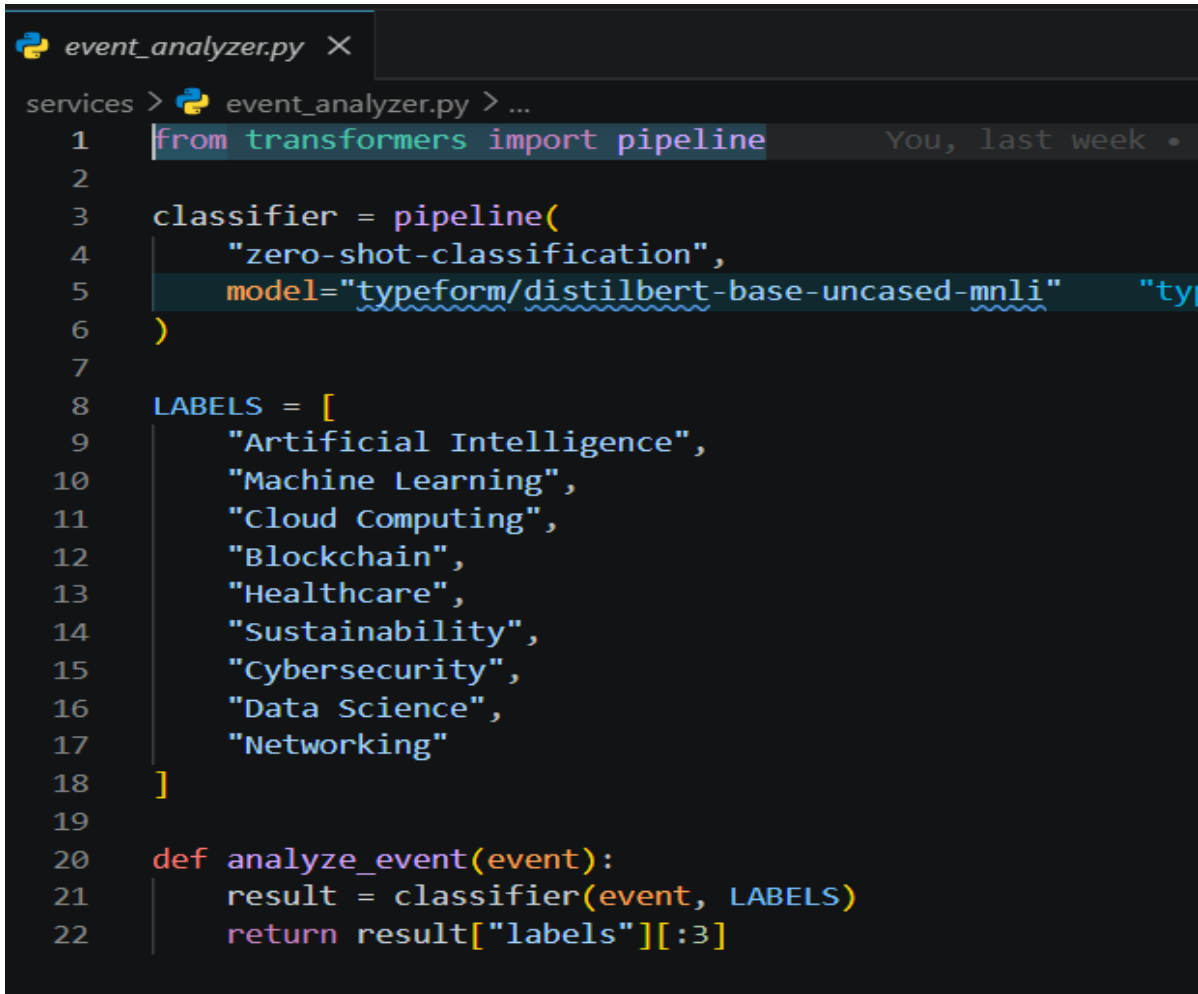
Integrate AI Models

The Personalized Networking Assistant integrates two AI technologies to provide intelligent networking assistance.

Google Gemini API

- Analyze the event description and user biography using the Google Gemini API.
- Generate personalized conversation starters based on the event context.
- Generate relevant networking topics and discussion ideas.

- Return AI-generated responses to the FastAPI backend for further processing.



```

event_analyzer.py X
services > event_analyzer.py > ...
1  from transformers import pipeline
2
3  classifier = pipeline(
4      "zero-shot-classification",
5      model="typeform/distilbert-base-uncased-mnli"
6  )
7
8  LABELS = [
9      "Artificial Intelligence",
10     "Machine Learning",
11     "Cloud Computing",
12     "Blockchain",
13     "Healthcare",
14     "Sustainability",
15     "Cybersecurity",
16     "Data Science",
17     "Networking"
18 ]
19
20 def analyze_event(event):
21     result = classifier(event, LABELS)
22     return result["labels"][:3]

```

Figure 3.2: Personalized Conversation Generation Using Google Gemini API

Integrate Wikipedia API

- Search Wikipedia using the generated networking topics.
- Retrieve reliable information related to the selected topic.
- Display verified facts to improve the quality of networking conversations.

Database Operations

- Store the user's event description.
- Store the user's biography.

- Save conversation history.
- Store generated networking topics.
- Save user feedback for future improvements.

Display Results

- Display AI-generated networking topics.
- Show verified information retrieved from Wikipedia.
- Display previous conversation history.
- Allow users to submit feedback through the application.

Epic 4: Frontend UI Using Streamlit

Epic 4 focuses on developing a user-friendly and responsive interface for the Personalized Networking Assistant. The frontend is built using Streamlit, providing an interactive interface that allows users to enter their profile details and event information. The application dynamically displays AI-generated conversation starters, networking topics, and verified facts in an organized layout

Story 1: Application Setup and Configuration

The frontend of the Personalized Networking Assistant was developed using Streamlit to provide a simple, interactive, and responsive user interface. Streamlit was selected due to its rapid development capabilities, seamless Python integration, and support for real-time data visualization.

The application is configured to communicate with FastAPI backend services through HTTP requests. During startup, the Streamlit application initializes required components, loads user interface elements, and establishes connections with backend APIs.

The configuration ensures smooth interaction between the user interface, NLP services, fact-checking functionality, and local data storage mechanisms.

Story 2: Input Section and Main Generation Flow

The application provides input fields for:

- User Biography
- Event Description
- Generate Conversation Button

When the user clicks the Generate Conversation Starters button, the input is validated and sent to the FastAPI backend, where Google Gemini generates networking suggestions.

Story 3: Results Display and Feedback System

The frontend displays:

- Personalized Conversation Starters
- Networking Topics
- Success/Error Messages

Users can also submit feedback, which is stored in the SQLite database for future improvements.

Story 4: Fact-Checking Section

The application includes a dedicated section that displays verified information retrieved from the Wikipedia API. This helps improve the reliability of AI-generated networking suggestions.

The system sends the query to the Fact Checker Service, which retrieves relevant information from Wikipedia and presents a concise summary.

Benefits of this feature include:

- Improving user confidence during networking conversations.
- Providing trusted background information.
- Supporting informed discussions at professional events.

The fact-checking interface is integrated seamlessly within the main application workflow.

Story 5: Conversation History View

Users can view previously generated networking conversations stored in the SQLite database. This allows them to revisit earlier networking sessions whenever required.

Information displayed may include:

- Event Description
- User Interests
- Generated Conversation Starters
- Timestamp of Generation

History records are retrieved from the local history storage system and displayed in a user-friendly format.

This functionality enables users to revisit successful conversation ideas and monitor their networking preparation over time.

Story 6: Feedback History View

The application allows users to view previously submitted feedback. This helps monitor user satisfaction and supports future improvements to the networking assistant.

The view typically includes:

- Generated Suggestion
- Feedback Type (Positive/Negative)
- Timestamp

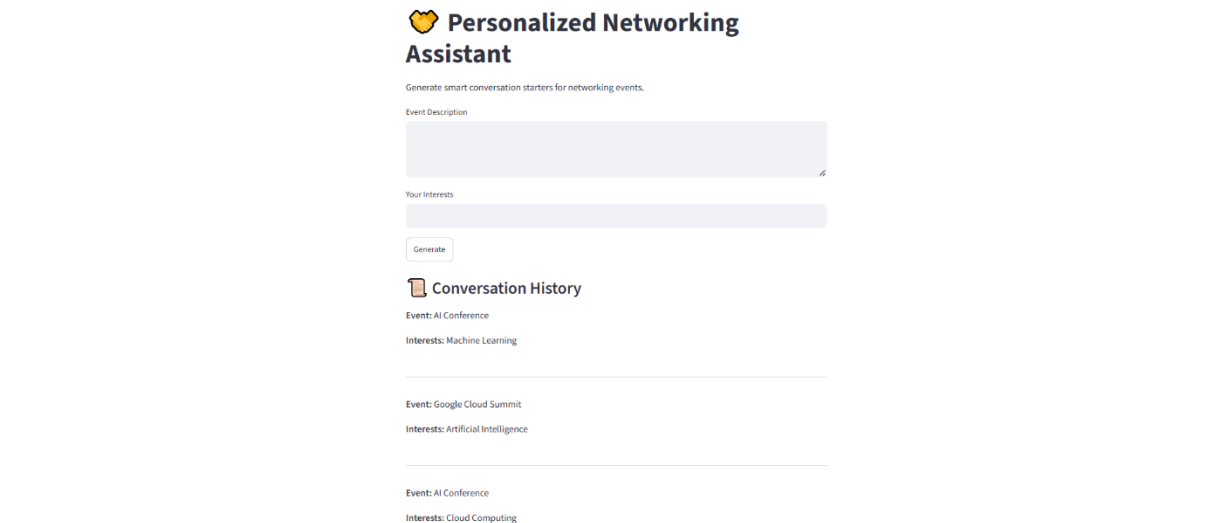
By maintaining a feedback history, the system supports transparency and future enhancement of recommendation quality.

The feature also demonstrates the application's ability to capture and utilize user interaction data effectively.

Designing and Developing the User Interface

Set Up the Streamlit Interface

- Develop the main Streamlit interface (streamlit_app.py) with a clean and intuitive layout.
- Create input fields for:
 - User Biography
 - Event Description
- Add a Generate Conversation Starters button.
- Display AI-generated networking suggestions in separate output sections.



The screenshot shows a web application titled "Personalized Networking Assistant" with a yellow heart icon. Below the title is a subtitle "Generate smart conversation starters for networking events." The interface has two input fields: "Event Description" and "Your Interests", both with light blue borders and a small blue icon on the right. Below these fields is a "Generate" button. Underneath the button is a section titled "Conversation History" with a document icon. This section displays three entries, each with an "Event" and "Interests" field. The first entry shows "Event: AI Conference" and "Interests: Machine Learning". The second entry shows "Event: Google Cloud Summit" and "Interests: Artificial Intelligence". The third entry shows "Event: AI Conference" and "Interests: Cloud Computing".

Figure 4.1: Streamlit User Interface

Design a Responsive Layout

- Organize input and output components using Streamlit containers and columns.
- Ensure the application adapts properly to different screen sizes.
- Display conversation starters, networking topics, and verified facts in clearly separated sections.

Create Output Components

- Conversation Starters: Display AI-generated conversation starters.
- Networking Topics: Show suggested discussion topics based on the event.
- Wikipedia Fact Verification: Display verified facts with their corresponding source links.

Dynamic Content Rendering

Handle User Input

- Collect the user biography and event description through the Streamlit interface.
- Validate user inputs before sending them to the FastAPI backend.

Render AI Responses Dynamically

- Receive responses from the FastAPI backend.
- Display:
 - Personalized conversation starters
 - Networking topics
 - Verified facts
- Update the interface instantly whenever the user submits new information.

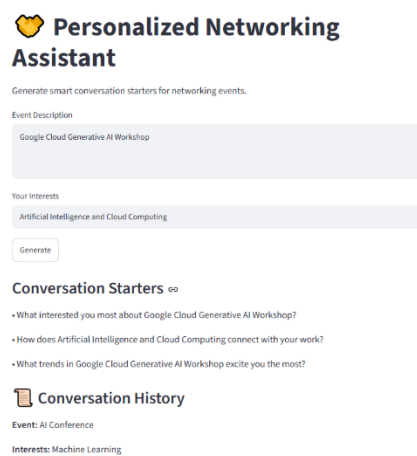


Figure 4.2: Generated Networking Suggestions

Maintain Application State

- Preserve previously generated results until the user submits a new request.
- Display appropriate success or error messages based on API responses.

Story 7: Streamlit State Management Implementation

The application uses Streamlit Session State to preserve generated networking suggestions, conversation history, user inputs, and fact-checking results during navigation. This improves the user experience by preventing data loss when users switch between different sections of the application or refresh the interface.

The implementation manages:

- Current user inputs
- Generated conversation starters
- Fact-check results
- Feedback selections
- Navigation state

Using session state prevents unnecessary recalculations and ensures that generated content remains available while users navigate through different sections of the application.

This approach improves responsiveness, enhances usability, and supports a more interactive frontend experience.

Epic 5: Testing and Local Deployment

Epic 5 focuses on deploying the **Personalized Networking Assistant** application and verifying that all modules work correctly in a local environment. During this phase, the development environment is configured, project dependencies are installed, and both the FastAPI backend and Streamlit frontend are executed successfully. The application is thoroughly tested to ensure that event analysis, networking topic generation, Wikipedia fact verification, conversation history, and user feedback operate as expected. Once the application is successfully tested locally, it can also be shared publicly using Ngrok for demonstration purposes.

Story 1: Testing Philosophy and Framework Selection

The application follows unit testing, integration testing, and manual testing approaches to ensure reliable performance. FastAPI TestClient is used for API testing while individual modules are verified independently.

The testing strategy focuses on verifying functional correctness, API behavior, data handling, and user interactions. The project uses **pytest** as the primary testing framework due to its simplicity, readability, and extensive support for automated testing. Additionally, **FastAPI TestClient** and **httpx** are used to test API endpoints and validate request-response behavior.

This approach helps identify issues early in development and ensures that the system performs as expected under different usage scenarios.

Story 2: Event Analyzer Service Testing

The Event Analyzer was tested using multiple event descriptions to verify accurate generation of networking suggestions.

The testing process ensured that:

- Event descriptions were successfully accepted.
- Relevant themes were extracted accurately.
- Invalid or empty inputs were handled gracefully.
- Response structures remained consistent.

Successful test execution confirmed that the service could reliably identify event topics and support subsequent conversation generation.

Story 3: Topic Generator Service Testing

The Topic Generator was tested using different generated networking suggestions to verify appropriate networking topic generation.

Test cases included:

- Standard networking events.
- Technology-focused conferences.
- Business and entrepreneurship events.
- Empty or incomplete user inputs.

The generated conversation starters were evaluated for relevance, clarity, and contextual alignment with the provided information. Testing confirmed that the service consistently produced useful networking prompts suitable for professional discussions.

Story 4: Fact Checker Service Testing

The Wikipedia Fact Checker was tested by retrieving information for multiple networking topics and validating successful API responses.

Testing objectives included:

- Verifying successful communication with the Wikipedia API.
- Confirming accurate retrieval of topic summaries.
- Handling unavailable or ambiguous search terms.
- Managing API errors and connection failures.

The service successfully returned concise factual information for supported topics and demonstrated reliable behavior under normal operating conditions.

Story 5: API Routes Testing with FastAPI TestClient

All FastAPI endpoints were tested using FastAPI TestClient to verify request validation, response generation, and database operations.

Testing objectives included:

- Verifying successful communication with the Wikipedia API.
- Confirming accurate retrieval of topic summaries.
- Handling unavailable or ambiguous search terms.
- Managing API errors and connection failures.

The service successfully returned concise factual information for supported topics and demonstrated reliable behavior under normal operating conditions.

Story 6: Running Tests and Interpreting Results

All unit tests executed successfully without major errors, confirming proper integration between the AI model, backend services, database, and frontend interface.

The testing process involved:

1. Executing test files using pytest.
2. Monitoring pass/fail results.
3. Reviewing assertion outputs.
4. Correcting identified issues.
5. Re-running tests after modifications.

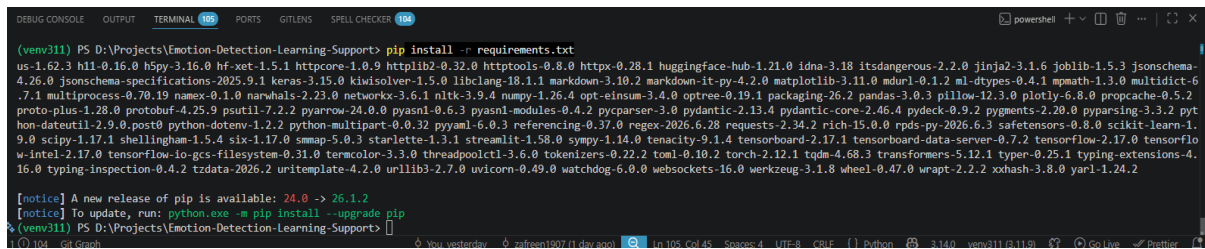
Test reports indicated that all major functionalities operated as expected. The successful completion of test cases provided confidence in the system's stability and readiness for deployment.

Story 7: Local Deployment

Before running the application, the development environment is configured by creating a Python virtual environment. This virtual environment isolates the project dependencies and ensures compatibility between the installed libraries. After activating the virtual environment, all required packages are installed using the **requirements.txt** file.

The application structure is verified to ensure that all project files, service modules, database files, and frontend components are available in their respective directories. The backend services responsible for AI-based event analysis, Wikipedia fact retrieval, database operations, and user feedback management are also checked before execution.

The FastAPI backend and Streamlit frontend are then prepared for execution by ensuring that all required dependencies have been installed successfully.



```

DEBUG CONSOLE OUTPUT TERMINAL 100 PORTS GIT LENS SPELL CHECKER 104 powershell
(venv311) PS D:\Projects\Emotion-Detection-Learning-Support> pip install -r requirements.txt
us-1.62.3 h11-0.16.0 h5py-3.16.0 hf-xet-1.5.1 httpcore-1.0.9 httplib2-0.32.0 httptools-0.8.0 httpx-0.28.1 huggingface-hub-1.21.0 idna-3.18 itsdangerous-2.2.0 Jinja2-3.1.6 joblib-1.5.3 jsonschema-
4.26.0 jsonschema-specifications-2025.9.1 keras-3.15.0 kiwisolver-1.5.0 libclang-18.1.1 markdown-3.10.2 markdown-it-py-4.2.0 matplotlib-3.11.0 mdurl-0.1.2 ml-dtypes-0.4.1 mmh3-1.3.0 multidict-6
.7.1 multiprocess-0.70.19 namex-0.1.0 narwhals-2.23.0 networkx-3.6.1 nltk-3.9.4 numpy-1.26.4 opt-einsum-3.4.0 optree-0.19.1 packaging-26.2 pandas-3.0.3 pillow-12.3.0 plotly-6.8.0 propcache-0.5.2
proto-plus-1.28.0 protobuf-4.25.9 psutil-7.2.2 pyarrow-24.0.0 pyasn1-0.6.3 pyasn1-modules-0.4.2 pycparser-3.0 pydantic-2.13.4 pydantic-core-2.46.4 pydeck-0.9.2 pygments-2.20.0 pyparsing-3.3.2 pyt
hon-dateutil-2.9.0.post0 python-dotenv-1.2.2 python-multipart-0.0.32 pyyaml-6.0.3 referencing-0.37.0 regex-2026.6.28 requests-2.34.2 rich-15.0.0 rpds-py-2026.6.3 safetensors-0.8.0 scikit-learn-1
9.0 scipy-1.17.1 shellingham-1.5.4 six-1.17.0 smmap-5.0.3 starlette-1.3.1 streamlit-1.58.0 sympy-1.14.0 tenacity-9.1.4 tensorboard-2.17.1 tensorboard-data-server-0.7.2 tensorflow-2.17.0 tensorflo
w-intel-2.17.0 tensorflow-io-gcs-filesystem-0.31.0 termcolor-3.3.0 threadpoolctl-3.6.0 tokenizers-0.22.2 toml-0.10.2 torch-2.12.1 tqdm-4.68.3 transformers-5.12.1 typer-0.25.1 typing-extensions-4
16.0 typing-inspection-0.4.2 tzdata-2026.2 uritemplate-4.2.0 urllib3-2.7.0 uvicorn-0.49.0 watchdog-6.0.0 websockets-16.0 werkzeug-3.1.8 wheel-0.47.0 wrapt-2.2.2 xxhash-3.8.0 yarl-1.24.2

[notice] A new release of pip is available: 24.0 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv311) PS D:\Projects\Emotion-Detection-Learning-Support>
  
```

Figure 5.1: Project Environment Setup and Dependency Installation

Story 8: Manual Testing and Verification

The Personalized Networking Assistant was manually tested using different user biographies and event descriptions. The application successfully generated networking topics, personalized conversation starters, verified facts through the Wikipedia API, stored conversation history in SQLite, and accepted user feedback. Invalid inputs were also tested to ensure proper validation and error handling.

Verification activities included:

- Entering event descriptions and interests.
- Generating conversation starters.
- Performing fact-check queries.
- Reviewing conversation history.
- Submitting user feedback.
- Testing navigation and interface responsiveness.

Manual testing confirmed that all features were accessible, functional, and user-friendly. The application successfully met the project requirements and delivered the intended networking assistance capabilities.

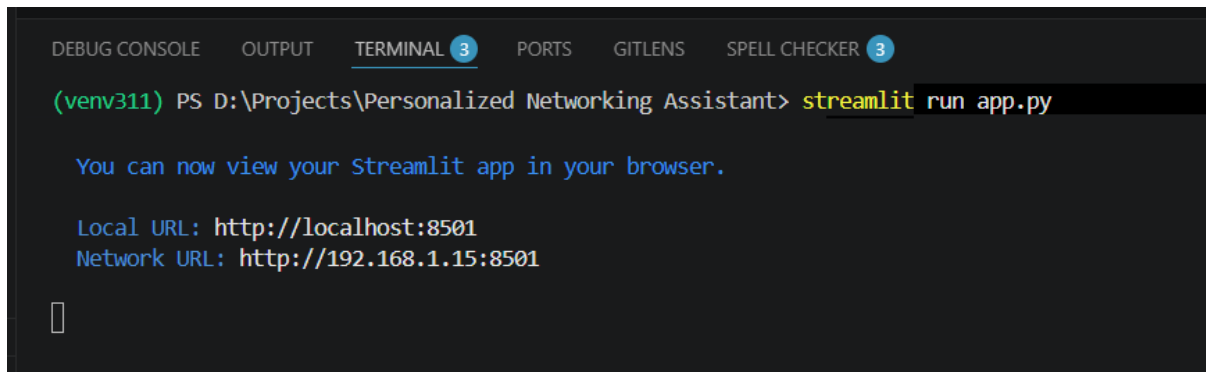
Conclusion of Testing Phase

Local Testing and Verification

After configuring the development environment, the FastAPI backend is started using the Uvicorn server, and the Streamlit frontend is launched. Once both services are running successfully, the application becomes accessible through the local browser.

Different event descriptions and user interests are entered into the application to verify the complete workflow. During testing, the system successfully accepts user input, analyzes event descriptions using the Google Gemini API, predicts relevant networking topics, retrieves verified information from Wikipedia, stores conversation history, and records user feedback.

The testing process also verifies that the application handles invalid or empty inputs appropriately and displays meaningful messages whenever necessary. Successful execution of these tests confirms that the application performs as intended in the local development environment.



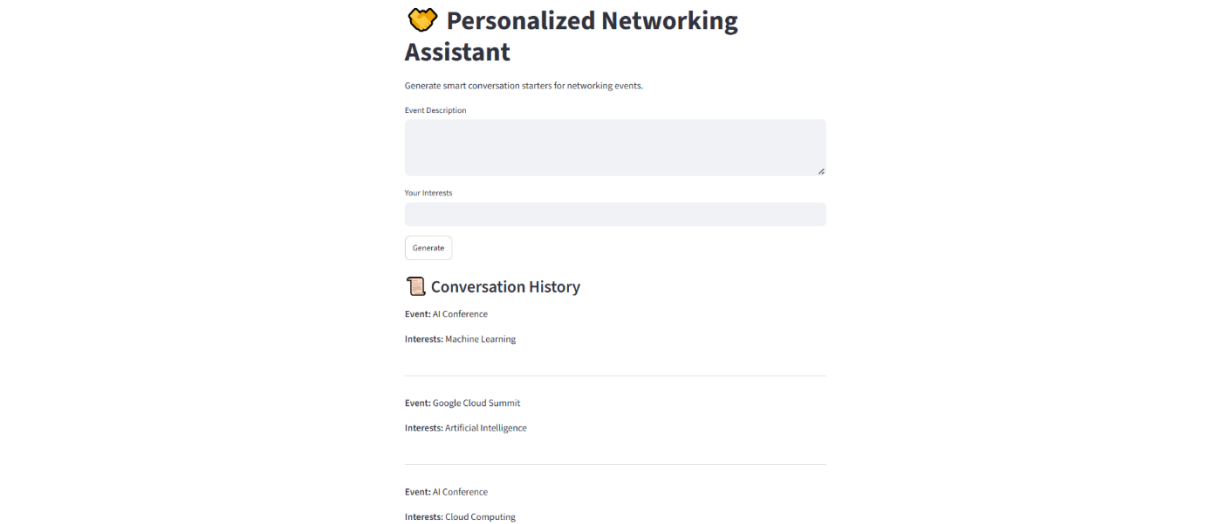
```
DEBUG CONSOLE  OUTPUT  TERMINAL 3  PORTS  GITLENS  SPELL CHECKER 3
(venv311) PS D:\Projects\Personalized Networking Assistant> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.15:8501
```

Figure 5.2: Personalized Networking Assistant Running Successfully

Exploring the Application Home Page



The screenshot shows the 'Personalized Networking Assistant' interface. It features a title with a heart icon, a subtitle 'Generate smart conversation starters for networking events.', and two input fields: 'Event Description' and 'Your Interests'. A 'Generate' button is positioned below the 'Your Interests' field. Underneath is a 'Conversation History' section with a book icon, listing three entries: 'AI Conference' with interest 'Machine Learning', 'Google Cloud Summit' with interest 'Artificial Intelligence', and another 'AI Conference' with interest 'Cloud Computing'.

Personalized Networking Assistant

Generate smart conversation starters for networking events.

Event Description

Your Interests

Generate

Conversation History

Event: AI Conference
Interests: Machine Learning

Event: Google Cloud Summit
Interests: Artificial Intelligence

Event: AI Conference
Interests: Cloud Computing

Figure 5.3: Personalized Networking Assistant Home Page

Description

The Home page provides a clean and interactive Streamlit interface where users can enter an user biography and event description. It also includes additional modules such as Conversation History, Wikipedia Fact Checker, and User Feedback. The simple layout allows users to access all major features from a single page.

Networking Suggestions Page

Personalized Networking Assistant

Generate smart conversation starters for networking events.

Event Description

Google Cloud Generative AI Workshop

Your Interests

Artificial Intelligence and Cloud Computing

Generate

Conversation Starters

- What interested you most about Google Cloud Generative AI Workshop?
- How does Artificial Intelligence and Cloud Computing connect with your work?
- What trends in Google Cloud Generative AI Workshop excite you the most?

Conversation History

Event: AI Conference

Interests: Machine Learning

Figure 5.4: AI-Generated Networking Suggestions

Description

After the user submits the user biography and event description, the application sends the request to the Google Gemini API, which generates personalized conversation starters and networking topics. Verified information retrieved from the Wikipedia API is displayed alongside the generated content. Users can also review previous conversation history and submit feedback through the application.

Conclusion

The deployment phase successfully demonstrates that the **Personalized Networking Assistant** operates efficiently using **FastAPI**, **Streamlit**, **Google Gemini API**, **Wikipedia API**, and **SQLite**. The application accurately analyzes event descriptions, identifies relevant networking topics, retrieves verified information, stores conversation history, and collects user feedback through an intuitive interface. The modular architecture ensures that the application can be easily extended in the future with enhanced prompt engineering, cloud deployment, user authentication, analytics dashboard, and integration with professional networking platforms. and improved networking recommendation techniques, cloud deployment, user authentication, and integration with professional networking platforms.